Parallelism Project: Is there parallelism among us? Parallel picture analysis

Parallelism and Distribution
Exam Requirements for the Practical Part of the Course
Academic Year 2024-2025

Lecturer: Elisa Gonzalez Boix
Assistant: Aäron Munsters
E-mail: pnd@dinf.vub.ac.be

1 Overview of the project

The goal of this project is to analyse an image of the collaborative Reddit Place project to find hidden characters in the picture. The project consists of three parts: the implementation of using Fork/Join parallelism, a performance evaluation, and a report that describes the implementation, evaluation and interpretation of the obtained results.

Deadline 25 May 2025 23:59 CEST. Late submissions will lead to a subtraction of points. Submissions later than 27 May 2025 23:59 CEST will be awarded 0. If you have an unexpected serious issue preventing you from submitting the project in time, please contact us beforehand.

Deliverable Hand in your solution via the assignment created in the course space on Canvas. To this end, create a ZIP file of your code and your report in PDF. Pay attention to the following:

- Your report should be named `<netid>-report-fj.pdf`[1]. The ZIP file should be named `<netid>-fj.zip`.
- If your project has dependencies that are not included in Java by default, add them in the ZIP file.
- Ensure that every source code file contains your netid (in comments) on the first line.

Evaluation You will be evaluated based on (1) the *correctness* of your implementation according to the specification, (2) the *correct use of the Java Fork/Join framework*, (3) the *quality of your code* (e.g., sufficient comments, clear variable names, well-structured, etc.), (4) the *quality of your report* (e.g., understandability of your solution, the interpretation of your results, etc.), and (5) the oral defence of your project during the exam period.

This assignment counts for 25% of your final grade, and is one of two assignments to be completed for the practical part of the course (the other one is the assignment on distribution). You will only be evaluated during this exam period if you hand in both assignments.

Before starting, carefully read the entire project assignment. On the Canvas course space, you can find some skeleton code (see Section 3) and guidelines for benchmarking (see Section 4).

Questions? Issues? Don't hesitate to contact us using pnd@dinf.vub.ac.be. Good luck!

Important!

[1] Your *NetID* (network identity) is a shortened representation of your name, such as e.g., *amunster*. It is **not** your student number.

The project is to be made **strictly individually** and **on your own**. This means that you must create your project on an independent basis and you must be able to explain your work, reimplement it under supervision, and defend your solution. Copying work (code, text, etc.) from or sharing it with third parties (e.g., AI tools, fellow students, websites, GitHub, etc.) is not allowed. Electronic tools will be used to compare all submissions with online resources and with each other, even across academic years.

The only code that may be copied is the code skeleton made available on Canvas (see Section 3).

Any action by a student that deviates from the instructions given and does not comply with the examination regulations is considered an irregularity. Plagiarism is also an irregularity. Plagiarism means the use of other people's work, adapted or otherwise, without careful acknowledgement of sources (cf. OER, Article 118$2). Plagiarism may relate to various forms of works including text, code, images, etc.

Any suspicion of plagiarism will be reported to the dean of faculty without delay. Both user and provider of such code will be reported and will be dealt with according to the plagiarism rules of the examination regulations (cf. OER, Article 118). The dean may decide on (a combination of) the disciplinary sanctions, ranging from 0/20 on the paper of the given programme unit to a prohibition from (re-)enrolling for one or multiple academic years (cf. OER, Article 118$5).

Contact us if you are in doubt as to whether or not something would be considered plagiarism.

2 Reddit Place & Among Us

Reddit Place (r/place) was a collaborative project and social experiment hosted on Reddit on April Fools' day 2017 and April Fools' day 2022[2]. The experiment consisted of an online canvas on which every Reddit user could add one pixel every 5 to 20 minutes. The result of the 2022 experiment is shown in Figure 1. As you can see, an incredibly complex piece of art emerged. A time-lapse of how this canvas was filled can be found here.

Upon closer inspection, the canvas reveals some unexpected elements: hidden in the entire canvas, one can find little 'amongi' (plural of 'amongus'), characters from the popular online game Among Us. These characters come in several different variants and colours.

The goal of this assignment is to analyse the canvas resulting from the Reddit place experiment to find the amongi that have been "hidden" in the picture – focussing on the variants shown in Figure 2 – (e.g., amongi are present in the green start button at the bottom left of the image). Moreover, a secondary goal is to analyse the colours of the amongi in the canvas, and to count how many amongi of each colour are present.

To help you start with this assignment, you can find some skeleton code on Canvas. This code contains already a sequential implementation to find all amongi within the canvas, and to count how many times each colour of amongus appears. Although finding amongi in a small canvas is still doable, the process becomes more time-consuming when the canvas grows. Hence, you need to parallelise this process using the Java Fork/Join framework according to the two phases outlined in this assignment (See Sections 3.1 and 3.2).

Your task is to provide a correct yet efficient implementation. To this end, you need to evaluate your implementation and analyse your results in the report.

[2] https://en.wikipedia.org/wiki/R/place

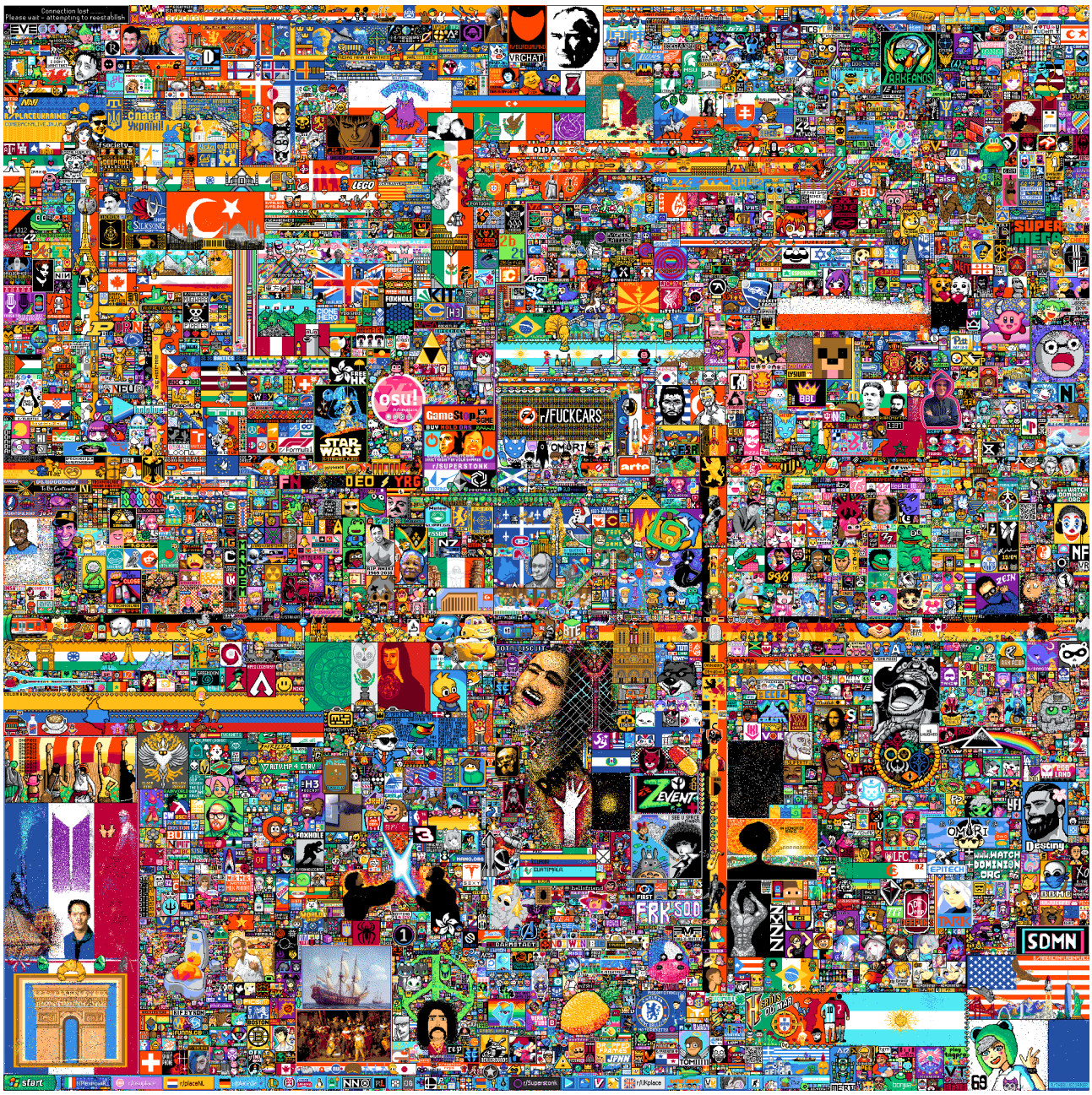

Figure 1: The result of the r/place experiment in 2022[3].

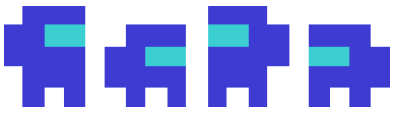

Figure 2: The amongi under scrutiny.

3 Implementation

On Canvas, together with this assignment, you can find a Java project that already contains the following code:

- `AmongiFinder`, an interface that needs to be implemented by classes that implement amongi finding and counting.
- `SequentialFinder`, a sequential implementation of the requested functionality.

- `Image`, a wrapper class for `BufferedImages` that allows you to read an image from a disk and query the colour (represented by an integer) of pixels.
- `Amongus`, a class that contains code to detect an amongus and its colour. The method `detect` checks whether a given coordinate is the top-left coordinate of an amongus, and returns `true` if this is the case.
- `Tests`, a class that contains some unit tests that can help you to verify the correctness of your implementation. You must *not* modify the given tests, but you are free to add more tests yourself.

You need to complete the classes `ParallelFinderGlobalHashMap` and `ParallelFinderSubtotals` using Fork/Join parallelism, according to the instructions below. The implementation consists of two phases, which both need to be evaluated.

3.1 Phase 1: Parallelising the x axis

As you can see, `SequentialFinder` traverses all coordinates in the canvas to see whether they are the top-left coordinate of an amongus. When doing so, it counts how many amongi have a specific body colour. The goal of this first phase is to parallelise the traversal over the x coordinates (the y coordinates are still to be traversed sequentially).

Implement the following two ways to count the colours of the amongi using a hashmap:

a **Local Hashmaps.** Keep the counters for all colours in a `HashMap<Integer, Integer>` that contains the subtotal of a given range of x coordinates. This result can be joined together with other results by adding all keys of one HashMap to the other and by ensuring that the results of keys appearing in both HashMaps are summed.

b **Global Hashmap.** Use *one* global Java's `ConcurrentHashMap` that is modified simultaneously by all Fork/Join tasks. Even though `ConcurrentHashMap` is thread-safe, watch out that you do not introduce race conditions in your implementation.

3.2 Phase 2: Parallelising the y axis

In addition to processing all x coordinates in parallel, the implementation can be further parallelized by also processing the y coordinates in parallel. Extend your implementation using the local HashMaps (Section 3.1a) to now also traverse the y coordinates in parallel (i.e., when you are handling a single x coordinate, the y coordinates should be handled in parallel). You do *not* have to implement Phase 2 using the global `ConcurrentHashMap` (described in Section 3.1b). You need to add a sequential cut-off to your implementation for parallelising the y axis.

3.3 Implementation requirements

Your implementation needs to adhere to the following non-functional requirements:

- Your implementation needs to be correct. The unit tests that are already provided alongside this project need to pass. You are not allowed to modify the given tests, but are free to add more tests yourself.
- Your implementation has to apply Fork/Join correctly. Use Fork/Join as seen in class (and as described in the course notes). Avoid expensive operations (dynamic memory allocations, frequent access to memory that is not in the cache,...). On the other hand, avoid micro-optimisations that decrease the readability of your code. Ensure that you add sufficient comments to your code, to improve its readability and understandability.

[3] Source: https://www.reddit.com/r/place/comments/txvk2d/rplace_datasets_april_fools_2022.

- To benchmark (see Section 4), your implementation of phase 2 will need to have a sequential cut-off. Pass the required cut-off to the constructor of `ParallelFinderSubtotals`.
- As mentioned in class, avoid the use of Java lambdas in your implementation, as these may affect the performance of your Fork/Join implementation. Also take into account the Java version available on Firefly (see Section 4.3.2).

4 Benchmarks

In order to evaluate the performance of your implementation, you will need to conduct benchmark experiments. First, you will benchmark the performance on your solution on own laptop. Then, you will repeat your benchmarks on Firefly, a powerful server located at the SOFT lab. In this section, we describe how to execute your benchmarks. In Section 5, you can find some questions that you will need to answer regarding your benchmarks and results. Together with this assignment, you will find the "Guidelines for Performance Evaluation" document to help you with benchmarking.

4.1 Dataset

To help you benchmark your solution, you can find several image files in the `images` folder of the Java project on Canvas. The largest image file, i.e., `place_23k_23k.png`, is an image of 23.000 by 23.000 pixels. Using the largest file will ensure that the runtime is sufficiently high so that you can clearly see speed-ups etc. In case you notice that your experiments take too long using this picture, you may use a smaller one. Please specify clearly in the report which image you used in your benchmarking. Make sure to run all your benchmarks on a given machine on the same image.

4.2 Benchmarking on your machine

To evaluate your implementations, you first have to execute your experiments on your own machine (e.g., on your laptop). For this, you must use a machine with at least 4 physical cores. In case you do not have this kind of machine, contact us via pnd@dinf.vub.ac.be. Using your machine, execute the experiments described in Sections 4.2.1 and 4.2.2.

4.2.1 Overhead and speed-up

You need to measure the overhead of both parallel implementations described in Section 3.1 (one using a global hashmap, and another in which local hashmaps are merged) and compare them. To have a fair comparison, ensure that both implementations process the y axis sequentially (i.e., put the threshold value of the implementation with the local hashmaps on `Integer.MAX_VALUE`). More concretely, you need to:

1. Compute the overhead of both implementations.
2. Compute the computational *and* application speed-up of the implementations for different values of `P` (e.g., 1, 2, 3, 4, 5, 6, 7, and 8 cores). Vary `P` so that you use *all* cores of your computer.

4.2.2 Optimal threshold

You also need to measure your implementation for Phase 1 using local HashMaps (cf. Section 3.1a) to find the optimal threshold value for your implementation for Phase 2 (cf. Section 3.2). More concretely, you need to:

1. Compute the overhead of your implementation for various values of the threshold. It is up to you to decide which threshold values to test. Choose them wisely!

2. Compute the application speed-up of your implementation for `P=<#cores of your computer>` and for the different threshold values.

4.3 Benchmarking on Firefly

Firefly is a powerful server with 128 logical cores and 128GB of memory. The server therefore is well-equipped to perform experiments that involve a lot of shared-memory parallelism.

Repeat the experiments which you performed on your own machine on Firefly. Where applicable, use all cores of Firefly. You do not need to repeat your experiments for *every* number of threads (1, 2, 3,..., 127, 128), but ensure that you get a picture of how your implementations scale to multiple cores.

4.3.1 Reserving Firefly

To run your benchmarks on Firefly, you need to book a *time slot*. A time slot is a period of 12 hours where you're the only active user on Firefly. Time slots will be available from **10 April until 11 May 2025**. There will be two slots per day, one from 09h until 21h, and one from 21h until 09h the next day. You only get **one slot** to run your benchmarks. **You can book your slot here. The deadline for registering a slot is 07 April 2025 23:59 CEST.**

You will have exclusive access to Firefly during the chosen slot. You need to strictly respect the bounds of your time slot. Otherwise, you possibly hamper the experiments of one of your fellow students! Keep the following in mind when using Firefly:

- You can use the server **only** in the context of this project.
- **Make sure that you are ready to start and run your benchmarks at the beginning of your slot.** This means that your code and benchmarking program should be ready, and you have already run your experiments locally so that you're sure that everything works correctly.
- At the beginning of your slot, check that there are no more experiments running of the previous student. If this is the case, try to reach out to your fellow student first, and otherwise send us an email to pnd@dinf.vub.ac.be.
- Calculate in advance (at the beginning of your slot) how much time your benchmarks will need so that you are sure that they can be completed within the given time bound (based on the number of experiments, the number of repetitions, the average runtime,...).
- Write your results to a file as soon as possible. This way, you don't lose data should something go wrong in the remainder of your program.
- In case you notice that your experiments have not yet finished at the end of your slot, you must end them. In case you already notice during benchmarking that your experiments will take too long, you can of course intervene. At the end of your slot, al your processes (including all your `tmux` sessions[4]) must be terminated. Using the command `tmux ls`, you can check whether you closed all tmux sessions. **Not respecting the bounds of your time slot may impact your score.**
- In case you experience technical difficulties during your time slot on the server (e.g., you cannot access the server (any more), another user is still using the server,...), do not hesitate to send an email to pnd@dinf.vub.ac.be. We will try to help you as soon as possible. In case required, we will try to give a backup slot, but we cannot promise that there will be one available.

To help you become acquainted with Firefly, a dedicated lab session in week 28 is scheduled. Its goal is to test your access to the server and become familiar with working on a server. The assignment of this WPO also overviews the guidelines regarding benchmarking on a server (cf. Section 4.4).

Keep into account that slots can fill up. Book your slot in due time.

[4] See the WPO on Firefly.

4.3.2 Specifications of Firefly

Finally, we list here Firefly's specifications:

- **Hostname:** firefly.vub.ac.be
- **Hardware:**
  - **Model:** Custom built
  - **Year:** 2021
  - **CPU:** AMD Ryzen™ Threadripper™ 3990X Processor (64 cores @ 2.9Ghz base, 4.3Ghz boost, 128 threads)
  - **RAM:** 128 GB DDR4-3200
  - **SSD:** 2x2TB nvme
- **Software:**
  - **OS:** Ubuntu 24.04.2 LTS
  - **Java:** openjdk version 14.0.2 (Java 14)

4.4 Guidelines for benchmarking

Benchmarking is hard. In order to help you do benchmarking in a scientific way, we have prepared a document titled "Guidelines for Performance Evaluation" available on Canvas together with this assignment. You are expected to follow and apply those guidelines (which will also be covered during the lab session in week 28). Certainly take into account the course-specific guidelines listed in Section 4 of the document.

Please keep in mind the following:

- For this project, you can benchmark using a fixed w (the number of warm-up iterations, e.g., 5) as done in the lab sessions. You are not expected to determine steady-state performance experimentally yourself.
- The guidelines document briefly discusses the "Java Microbenchmark Harness (JMH)". This framework is often used by researchers to execute microbenchmarks on the JVM, as it helps to avoid frequent pitfalls. You do *not* need to use this framework for your project. In case you decide to use JMH, and it is correctly used, you will be rewarded with a bonus.

5 Report

The report consists of two parts: a brief description of your implementation(s), and the description of the evaluation hereof on your own machine and on Firefly. For your report, carefully take into account the guidelines for reporting your experiments and results discussed in the "Guidelines for Performance Evaluation" document. The length of your report, excluding figures, is maximally 8 pages. The report must have the following structure:

1. **Implementation.** Briefly describe your implementation and the important decisions so that one can understand your solution without delving into the code. To this end, answer the following questions: Which classes implement Phase 1 and which classes implement Phase 2? Did you use any of the Fork/Join patterns seen in class (map, reduction, prefix, pack, parallel sorting) in your implementation? (max. 1 page).
2. **Evaluation.** Discuss the performed benchmarks and obtained results. It's important to look at your results critically and analytically, i.e., for **all** questions, you should **explain** what your

results mean (w.r.t. your program) and why you see these results. Report your results using scientifically valid graphical representations (e.g., use box plots, histograms,...), following the guidelines in the aforementioned document. In your text, clearly indicate corresponding figures. If applicable, mention your expectations w.r.t. obtained results.

Discuss for every question below first your results on your local computer and then the results of your benchmarks on Firefly. **Respond to every question separately, using a separate section that has been numbered accordingly.**

2.1. Experiments and experimental set-up

In this section, discuss all information relevant w.r.t. the execution of your benchmarks, following the guidelines in the aforementioned document. Make sure to clearly explain what your experimental set-up is (e.g., include the specification of **each** machine used according to the guidelines document), how you executed your experiments, and what you measured.

2.2. Overhead and speed-up

In this section, discuss the overhead and speed-up of your implementations based on your experiments from Section 4.2.1.

1. What is the overhead of your implementations? Is this overhead acceptable? What causes this overhead?
2. What are the speed-ups of your implementations? Are these speed-ups higher or lower than expected? Why?
3. What speed-up is larger: the computational or application speed-up? Is this what you expect? Why?
4. How well do your implementations scale? Why do they (not) scale well?
5. Based on your experiments, which of both implementations is the best? Why?

2.3. Optimal threshold

In this section, use your results for the experiments from Section 4.2.2 to investigate the influence of a threshold.

1. How does the threshold influence the overhead and speed-up of your implementations? What happens when you make the threshold larger or smaller? Explain.
2. Suppose you would need to determine a threshold naively (i.e., **without** benchmarking). How would you do this? What would you consider?
3. Based on your measurements, what is the best threshold? Why?

2.4. Implementation characteristics

In this section, you will analyse your implementation based on its asymptotic bounds. You only need to do this for your implementation with the local hashmaps (i.e., the parallelisation of phase 1 and 2 using local hashmaps). Clearly show and explain your calculation and all symbols used. Assume merging two hashmaps is O(1)

1. What is the `work` of the computation?
2. What is the `span` of the computation?
3. How many tasks are created by your implementation?